

6. ROS2 Nodes

Note: The accompanying reference video uses the ROS Humble version, for reference only

1. Node Overview

During communication, regardless of the method used, the construction of communication objects depends on Nodes. In ROS2, each node generally corresponds to a single functional module (for example: the LiDAR driver node may be responsible for publishing LiDAR messages, the camera driver node may be responsible for publishing image messages). A complete robot system may consist of many nodes working together. A single executable file (C++ program or Python program) in ROS2 can contain one or more nodes.

2. Node Creation Process

1. Create program file
2. Import related ROS libraries
3. Write node functionality
4. Write configuration files
5. Build and run

3. Hello World Node Example

Here we use a Python package as an example for explanation.

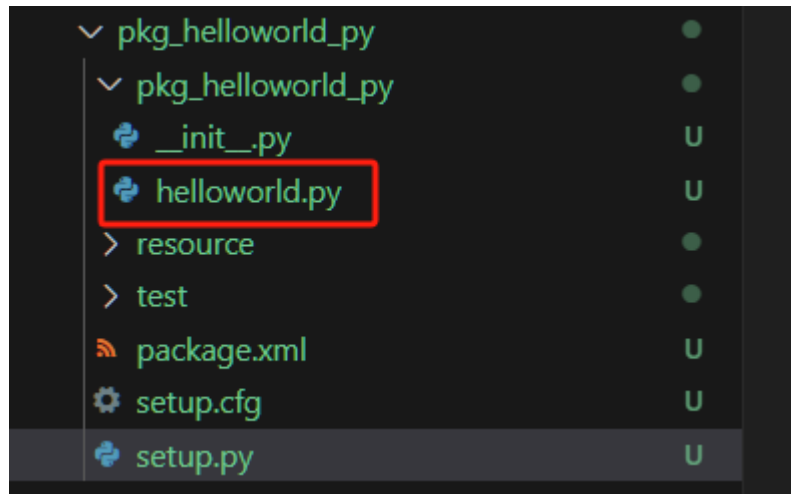
3.1. Create Python Package

- Replace workspace with your actual workspace path:

```
cd workspace/src
ros2 pkg create pkg_helloworld_py --build-type ament_python --dependencies rclpy --node-name helloworld
```

3.2. Write Code

After executing the above command, pkg_helloworld_py will be created, and a helloworld.py file will be generated for writing the node:



Delete the original helloworld.py code and write the following code:

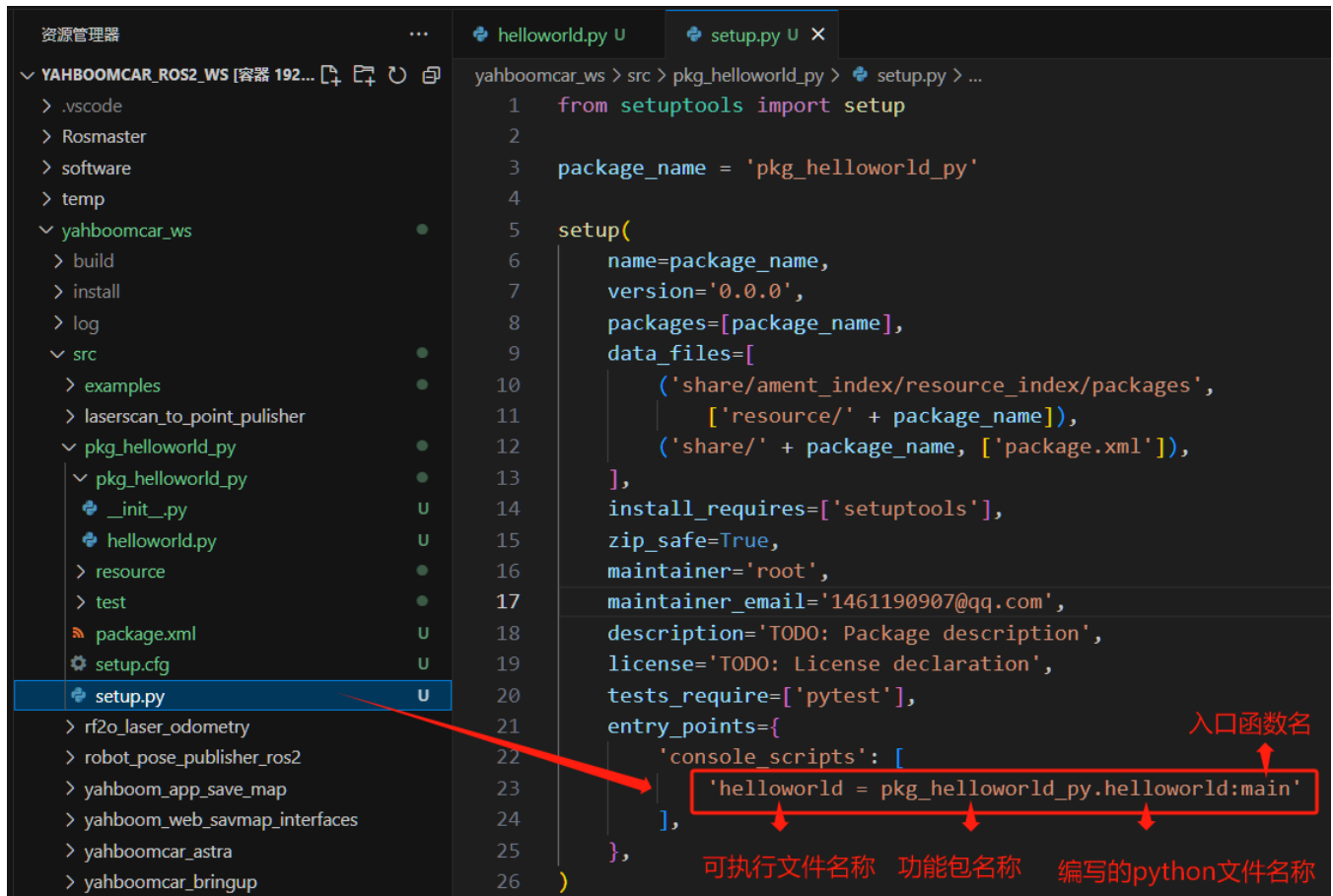
```
import rclpy                                # ROS2 Python interface library
from rclpy.node import Node                 # ROS2 Node class
import time

"""
Create a HelloWorld node that outputs "hello world" log during initialization
"""

class HelloWorldNode(Node):
    def __init__(self, name):
        super().__init__(name)              # ROS2 node parent class initialization
        while rclpy.ok():                   # Check if ROS2 system is running
            normally
                self.get_logger().info("Hello world") # ROS2 log output
                time.sleep(0.5)                 # Sleep to control loop timing

def main(args=None):
    # ROS2 node main entry point
    rclpy.init(args=args)                   # ROS2 Python interface initialization
    node = HelloWorldNode("helloworld")      # Create ROS2 node object and initialize
    rclpy.spin(node)                        # Loop waiting for ROS2 exit
    node.destroy_node()                     # Destroy node object
    rclpy.shutdown()                        # Shutdown ROS2 Python interface
```

After completing the code, you need to configure the package's build options so the system knows the entry point for the Python program. Open the package's setup.py file and add the following entry point configuration:



3.3. Build Package

- Build the package:

```
colcon build --packages-select pkg_helloworld_py
```

- Refresh environment variables in the workspace:

```
source install/setup.bash
```

3.4. Run Node

```
ros2 run pkg_helloworld_py helloworld
```

Upon successful execution, you can see the effect of continuously printing "Hello World" in the terminal:

```
root@unbutu:~/yahboomcar_ros2_ws/yahboomcar_ws# ros2 run pkg_helloworld_py helloworld
[INFO] [1698050077.167925417] [helloworld]: Hello World
[INFO] [1698050077.669687813] [helloworld]: Hello World
[INFO] [1698050078.170830157] [helloworld]: Hello World
[INFO] [1698050078.672503205] [helloworld]: Hello World
[INFO] [1698050079.174207487] [helloworld]: Hello World
[INFO] [1698050079.675991674] [helloworld]: Hello World
[INFO] [1698050080.177530190] [helloworld]: Hello World
[INFO] [1698050080.679924285] [helloworld]: Hello World
[INFO] [1698050081.182654103] [helloworld]: Hello World
[INFO] [1698050081.685459571] [helloworld]: Hello World
[INFO] [1698050082.188045608] [helloworld]: Hello World
```